In both cases, you're running the tests through JUnit.

One way you can configure the automated build to run these tests in separate parallel threads is to use the `<forkCount>` option, as shown here:

```
<plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-surefire-plugin</artifactId>
    <version>2.16</version>
    <configuration>
        <forkCount>4</forkCount>
        <reuseForks>true</reuseForks>
        <includes>
            <include>com/bddinaction/chapter12/*.java</include>
        </includes>
    </configuration>
</plugin>
```

Run the tests in four parallel JVM processes.

Don't create a new process for each new test.

Run all of the acceptance test stories.

This runs each test class in a separate JVM process. You can also use JUnit's built-in `<parallel>` option, but this runs all of the tests in parallel threads within a single JVM process, which is less robust and better suited to running unit tests in parallel.

The `<forkCount>` option will also work fine with the Maven Failsafe plugin, a popular alternative Maven plugin used for integration testing.[1]

Similar approaches can work with any tool that uses a JUnit-based test runner, though with some caveats. If you're using Cucumber-JVM, you can configure Cucumber-JVM test suites similar to the Thucydides test runners shown previously to run a subset of the Cucumber feature files. For example, the following class will run all of the feature files with the `@earning_points` tag:

```
@RunWith(Cucumber.class)
@Cucumber.Options(tags = {"@earning_points"},
                  format = {"json:target/cucumber/earning_points.json"})
public class EarningPoints {}
```

Run only scenarios with this tag.

Store the reports in JSON format in this file.

Similarly, the following class will only run features related to Authentication:

```
@RunWith(Cucumber.class)
@Cucumber.Options(tags = {"@authentication"},
                  format = {"json:target/cucumber/authentication.json"})
public class Authentication {}
```

Note how each of these test suites writes the test results to a different file. This ensures that the test results can be produced independently, but it means you'll need to merge the results from all of these JSON files after the tests have finished. If you're using a CI tool with Cucumber integration, such as Jenkins or Bamboo, this can be easily done directly from within the CI build job itself.

Not all BDD tools offer this level of support for parallel testing, and even when this is the case, parallel testing on a single machine won't scale beyond a certain point. For

[1] See "Fork Options and Parallel Test Execution" on Apache's "Maven Failsafe Plugin" page: http://maven.apache.org/surefire/maven-failsafe-plugin/examples/fork-options-and-parallel-execution.html.